

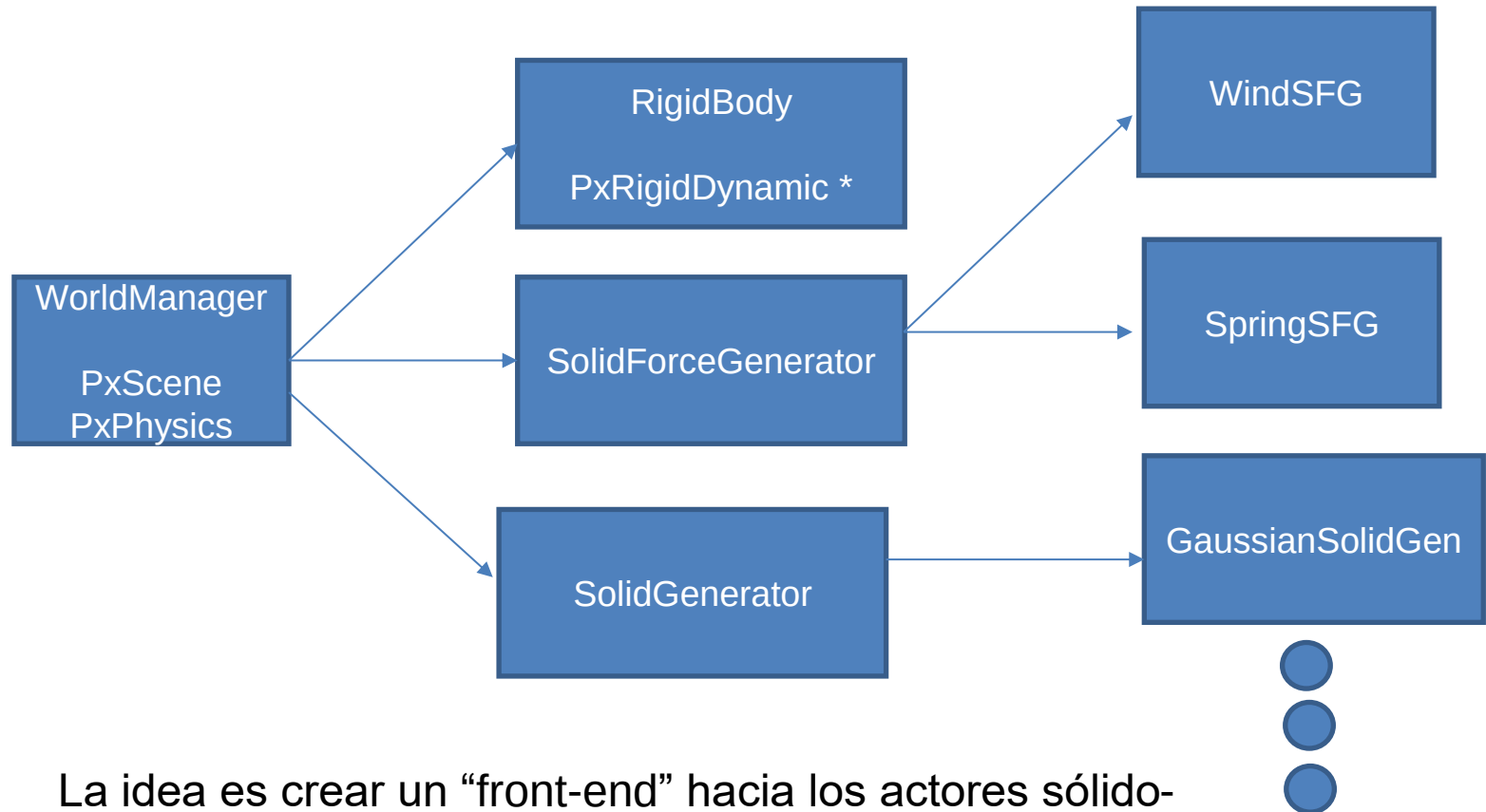
Implementación de física de estado sólido

R. Fernandez
L. García
D. Alejo

Motor de sólido Rígido de Physx. Introducción

- ☐ Introducido en 2008
- ☐ Tiene dos partes:
 - ☐ SDK para realizar simulaciones físicas
 - ☐ Driver para ejecutarlas con tarjetas gráficas
- ☐ SDK: Usamos la version 3.4
 - ☐ De Código abierto:
 - ☐ <https://github.com/NVIDIAGameWorks/PhysX-3.4>
 - ☐ <https://github.com/NVIDIA-Omniverse/PhysX/tree/release/104.0/physx>
- ☐ Características
 - ☐ Gestiona sistemas de sólidos rígidos
 - ☐ Respuesta a colisiones
 - ☐ Addons de simulación de tejidos, conducción, física partículas...

Diagrama de clases: incorporando sólidos rígidos



La idea es crear un “front-end” hacia los actores sólido-rígidos de Physx que nos permita

- Generar nuevos sólidos que interactúen con la escena
- Aplicar fuerzas sobre dichos sólidos

Inicialización de Physx

- ❑ El primer paso es definir una instancia de PxFoundation.
- ❑ PX_FOUNDATION_VERSION es una macro definida dentro de physx.
- ❑ El allocator callback y error callback se pueden definir de forma personalizada para cada aplicación pero physx ya proporciona una implementación por defecto.

```
using namespace physx;
```

```
PxDefaultAllocator      gAllocator;  
PxDefaultErrorCallback  gErrorCallback;
```

```
// Required for initialization
```

```
gFoundation = PxCreateFoundation(PX_FOUNDATION_VERSION, gAllocator, gErrorCallback);
```

Inicialización de Physx

- ❑ Definimos el objeto gPhysics que contiene los métodos de alto nivel de Physx.
- ❑ PXToleranceScale nos permite cambiar la escala de la simulación en tamaño y velocidad. Cambiar la velocidad aquí equivale a escalar la velocidad como hicimos en la práctica 2. Cambia toda la simulación. Recomendado dejar por defecto.

```
gPvd = PxCreatePvd(*gFoundation);  
PxPvdTransport* transport = PxDefaultPvdSocketTransportCreate(PVD_HOST, 5425, 10);  
gPvd->connect(*transport, PxPvdInstrumentationFlag::eALL);
```

```
gPhysics = PxCreatePhysics(PX_PHYSICS_VERSION, *gFoundation, PXToleranceScale(), true, gPvd);
```

Inicialización Physx con sólido rígido

- ❑ El primer paso para definir nuestros sólidos rígidos es generar una escena (pxScene).
- ❑ Una escena es una simulación de un conjunto de elementos (actores) que pueden interaccionar entre ellos.
- ❑ Puede haber varias escenas, pero cada actor solo puede pertenecer a una de ellas.

```
// For Solid Rigid ++++++
PxSceneDesc sceneDesc(gPhysics->getTolerancesScale());
sceneDesc.gravity = PxVec3(0.0f, -9.8f, 0.0f);
gDispatcher = PxDefaultCpuDispatcherCreate(2);
sceneDesc.cpuDispatcher = gDispatcher;
sceneDesc.filterShader = contactReportFilterShader;
sceneDesc.simulationEventCallback = &gContactReportCallback;
gScene = gPhysics->createScene(sceneDesc);
```

Generación de actores

- ❑ Una vez que tenemos definida nuestra escena vamos a añadir actores a ella.
- ❑ Hay dos tipos de actores
 - ❑ [PxRigidStatic](#): Elementos fijos en la escena.
 - ❑ [PxRigidDynamic](#): Elementos dinámicos en la escena (pueden cambiar su posición)

Generación de actores estáticos

- ☐ Vamos a empezar por los elementos fijos de la escena.
- ☐ Para definir un elemento en la escena necesitamos definir una posición y una forma.

```
// Generar suelo
PxRigidStatic* Suelo = gPhysics->createRigidStatic(PxTransform({ 0,0,0 }));
PxShape* shape = CreateShape(PxBoxGeometry(100, 0.1, 100));
Suelo->attachShape(*shape);
gScene->addActor(*Suelo);
```

- ☐ *addActor* añade el elemento a la escena gscene.

```
// Pintar suelo
RenderItem* item;
item = new RenderItem(shape, Suelo, { 0.8, 0.8,0.8,1 });
```

- ☐ Seguimos necesitando renderizar el elemento

Generación de actores dinámicos

- ❑ Vamos a añadir actores dinámicos.
- ❑ En el caso de un actor dinámico podemos añadirle, entre otras cosas, una velocidad lineal y angular.

```
// Añadir un actor dinámico
PxRigidDynamic* new_solid;
new_solid = gPhysics->createRigidDynamic(PxTransform({ -70, 200, -70 }));
new_solid->setLinearVelocity({ 0, 5, 0 });
new_solid->setAngularVelocity({ 0, 0, 0 });
PxShape* shape_ad = CreateShape(PxBoxGeometry(5, 5, 5));
new_solid->attachShape(*shape_ad);
```

Generación de actores dinámicos

- ❑ Es muy importante definir también la distribución de masas.

```
PxRigidBodyExt::updateMassAndInertia(*new_solid, 0.15);  
gScene->addActor(*new_solid);
```

- ❑ Usando *updateMassAndInertia* el cálculo de la distribución de masas y momentos de inercia es automático.
- ❑ *updateMassAndInertia* supone una **distribución uniforme** con la **densidad** especificada (en este caso 0.15 kg/m³).
- ❑ También se puede hacer usando la siguiente función fijando a mano el **tensor de inercia**.

```
new_solid->setMassSpaceInertiaTensor({ size.y * size.z, size.x * size.z, size.x * size.y });
```

Generación de actores dinámicos

- ❑ El código completo, incluyendo el renderizado quedaría algo tal que así

```
// Añadir un actor dinámico
PxRigidBody* new_solid;
new_solid = gPhysics->createRigidBody(PxTransform({ -70,200,-70 }));
new_solid->setLinearVelocity({ 0,5,0 });
new_solid->setAngularVelocity({ 0,0,0 });
PxShape* shape_ad = CreateShape(PxBoxGeometry(5,5,5));
new_solid->attachShape(*shape_ad);
PxRigidBodyExt::updateMassAndInertia(*new_solid, 0.15);
gScene->addActor(*new_solid);
```

```
// Pintar actor dinámico
RenderItem* dynamic_item;
dynamic_item = new RenderItem(shape_ad, new_solid, { 0.8, 0.8,0.8,1 });
```

Ejemplo 1: Ejecución del código anterior

- ❑ El siguiente video muestra el resultado de añadir el código anterior a la función `InitPhysics` de nuestro esqueleto.



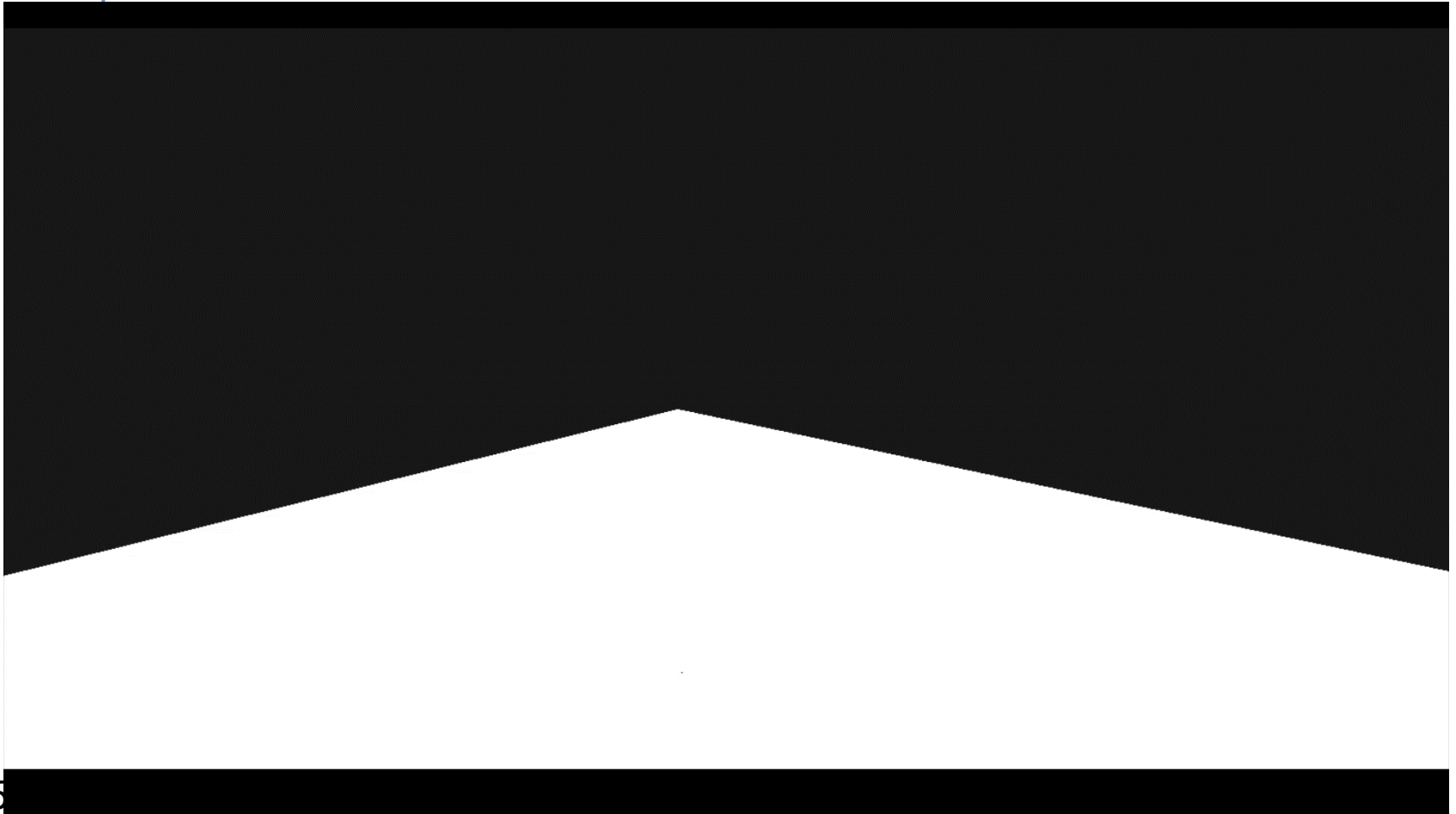
Ejemplo 2: Colisión cubos distinta masa

- ❑ Añadiendo un segundo cubo con una densidad mucho mayor encima



Ejemplo 3: Colisión cubos distinta masa

- ❑ Añadiendo un segundo cubo con una densidad mucho menor encima



PxRigidBody: métodos importantes

- ❑ Relacionados con la masa: **set/getMass**
- ❑ Tensor de inercia: **setMassSpaceInertiaTensor**
- ❑ Velocidad lineal: **set/getLinearVelocity**
- ❑ Velocidad angular: **set/getAngularVelocity**
- ❑ Para gestionar dinámica (generadores fuerza):
 - ❑ **addForce()**
 - ❑ **addTorque()** (opcional)
- ❑ **Attach Shape** → para definir la forma y el material del objeto
 - ❑ PxMaterial tiene información de:
 - ❑ Coeficientes de rozamiento estático y dinámico
 - ❑ Coeficiente de restitución ante choques

Resolución de colisiones: acciones extra

PxSimulationEventCallback

- ❑ Existe la clase `PxSimulationEventCallback` que debemos extender para hacer acciones cuando se producen ciertos eventos de simulación:
 - ❑ Se despierta o duerme un actor
 - ❑ Se rompen restricciones
 - ❑ **Se producen contactos o colisiones**
- ❑ Para añadir comportamientos ante colisión, debemos capturar esos eventos mediante funciones "Callback" que se llamarán automáticamente cuando se produzcan.

Resolución de colisiones: acciones extra

- ❑ Esto ya lo tenéis hecho y el callback ante colisión redirige a la función **onCollision**, dentro de main. Para añadir comportamiento:
 - ❑ Buscaremos los objetos en el WorldManager
 - ❑ Si son objetos importantes, efectuaremos acciones en la lógica del juego.

```
void onCollision(physx::PxActor* actor1, physx::PxActor* actor2)
{
    PX_UNUSED(actor1);
    PX_UNUSED(actor2);
}
```

PxRigidBodyDynamic: métodos importantes

- ❑ **PxRigidBodyDynamic** hereda de PxRigidBody. Añade:
 - ❑ Damping linear/angular
 - ❑ Velocidades máximas y mínimas
 - ❑ Gestión de colisiones

PxPhysics & PxScene: Integración

- ☐ El motor de Physx se encarga de llevar a cabo los pasos de integración
- ☐ Nos deja elecciones de los algoritmos a aplicar en cada fase
 - ☐ Detección de colisiones fina/gruesa
 - ☐ Distintos impulsos ante colisiones (coeficiente de restitución)
 - ☐ Número máximo de iteraciones a realizar
 - ☐ Generación de rozamiento en superficies en contacto
- ☐ Read the docs:
 - ☐ <https://nvidia-omniverse.github.io/PhysX/physx/5.1.0/docs/Simulation.html#simulation-loop>

PxPhysics: Limpieza

- ❑ Como vimos anteriormente, la clase Physx es propietaria de los punteros a los objetos que se crean en él
- ❑ Dichos punteros se liberan automáticamente cuando el objeto Physx se destruye

